

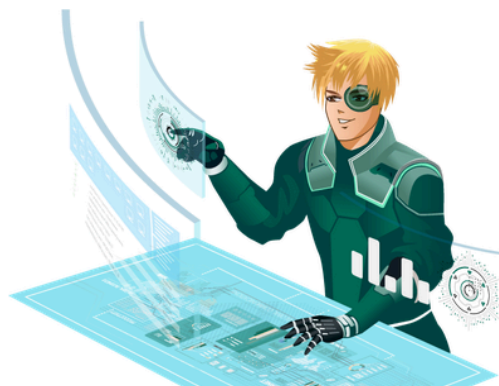
Evading AV/EDR with Cobalt strike

🌐 logan-goins.com/2024-02-03-CS

Logan Goins

Software for Adversary Simulations and Red Team Operations

[Adversary Simulations](#) and [Red Team Operations](#) are security assessments that replicate the tactics and techniques of an advanced adversary in a network. While penetration tests focus on unpatched vulnerabilities and misconfigurations, these assessments benefit security operations and incident response.



Cobalt strike is the de-facto adversary emulation framework. Cobalt strike combines power, flexibility, and customizability, being so prevalent in today's threat landscape that most EDR and XDR are really just "Cobalt strike" detectors. So anyone could see how Cobalt strike payloads are easily some of the most signed software ever, and still through the customizability and "malleability" of Cobalt strike it can still fly under the radar even when pinned against a whole host of defenses. This makes it a fan favorite for simulated and non-simulated adversaries.

The Eternal Game of Cat & Mouse

Adversaries are always exploiting defenses, defenders are always mitigating these vulnerabilities, and adversaries are always bypassing those mitigations. This cycle will presumably go on forever, making offensive security tooling development a "Cat and Mouse Game", with the developers of these tools emulating adversary techniques and discovering new ways to get around mitigations. This ensures that an organization will already have a report sitting on a SOC analyst's desk before the actual adversaries decide to use these techniques.

This same thing is as true for Cobalt strike development as everything else. Taking a look back at Cobalt strike's history, and history of adversary emulation in general we can see a pattern form. From PowerShell Empire's rise to power, to nowadays adversaries relying mostly on execution of .NET assemblies in memory, it has always been this way. Which is why it's important to remember that these techniques used to bypass AV will not work forever, and the field will move past these techniques through a circular motion of one-upmanship.

Malleable C2 Profile Configuration

If we want to bypass Defender, first we have to utilize a custom C2 profile, which allow us a number of options for customizing our staging, and beacon payloads. We can specify a C2 profile when starting up our teamserver.

An easy way to throw off signatures is to strip strings out of your beacon payload, this can be done in the transform blocks and by replacing commonly signed strings with whitespace:

```
transform-x64 {
    prepend "\x90\x90\x90\x90\x90\x90\x90\x90\x90";
    strrep "This program cannot be run in DOS mode" "";
    strrep "ReflectiveLoader" "";
    strrep "beacon.x64.dll" "";
    strrep "beacon.dll" "";
    strrep "msvcrt.dll" "";
    ...
}
```

We can also add a post-ex block to do tons of important things for our post exploitation efforts:

```
post-ex {

    set spawn_to_x86 "%windir%\syswow64\notepad.exe";
    set spawn_to_x64 "%windir%\sysnative\notepad.exe";

    set obfuscate "true";
    set smartinject "true";
    set amsi_disable "true";

}
```

1. The “spawn_to” configuration tells beacon that when it uses modules such as “execute-assembly”, to start the specified program up as a child process, inject the capability into it, receive output over a named pipe and terminate it. This is called the fork&run process.
2. “obfuscate” which when set to true loads Beacon into memory without its DLL headers which would be an IOC in memory, and heavily signed
3. And finally “amsi_disable”, which uses a hardware breakpoint technique to bypass the AMSI memory scanner to evade detection when executing programs in memory like with “execute-assembly”

There are some other options to look into, such as the “allocator” configuration setting which allows you to set a different Win32API call for shellcode loading, I set mine to

“NtMapViewOfSection”. The last special mention is the “userwx” setting, which when set to false allocates memory for the Beacon DLL as RW/RX instead of RWX, which is significantly less suspicious.

The Artifact Kit

The Artifact kit is used to help expand the customizability of Cobalt strike to a new level, allowing operators to manually modify Beacon and module source code. We can use the Artifact kit, along with ThreatCheck, and a Reverse Engineering (RE) tool like Ghidra, to identify exactly where Windows Defender is flagging the malicious bytes and change the Beacon source code, bypassing signature based detection.

I mainly modified the named pipe bypass version of Beacon, and it took quite a long time with numerous changes, so I'll just note the main changes, being the shellcode payload XOR decoder in the patch.c file, and the hardcoded named pipe title in the bypass-pipe.c file. Changing variable names and inserting useless function calls completely modified the compiled bytes, giving me another piece of the puzzle for AV bypass.

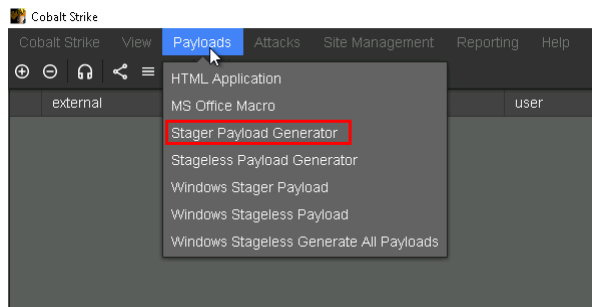
Shellcode Loading the Cobalt strike Stager Into a Callback

I know what you're thinking, “isn't the CS stager just a shellcode loader which requests the full beacon stage (shellcode) from the C2 server and loads it? Making us load shellcode to load shellcode?”, and yes, you would be correct! The problem is the signed stager and staging protocol, and Defender likes to peep on suspicious processes and our new stager is one of them. If we use something a little less signed and maybe some light sandbox bypasses we can easily get around this. Windows Defender is significantly better at scanning files on disk, so if we only execute the full beacon payload in memory it'll be more difficult to detect.

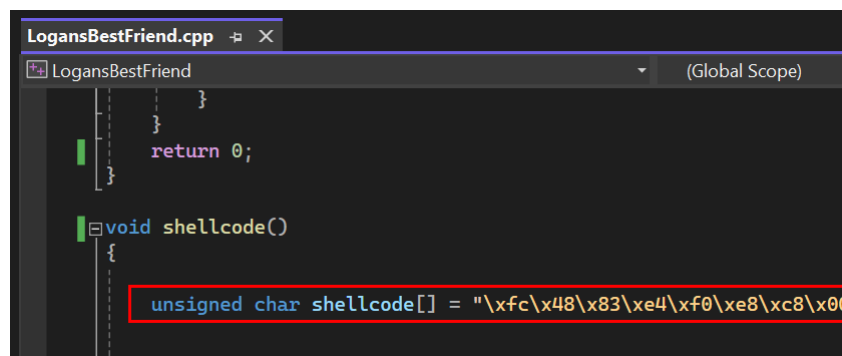
For this, I've decided to use a shellcode loader from WhiteKnightLabs called GregsBestFriend that I've modified for more practical applications which I call “LogansBestFriend”.

The source for compilation with Visual Studio can be found on my GitHub:
[LogansBestFriend](#)

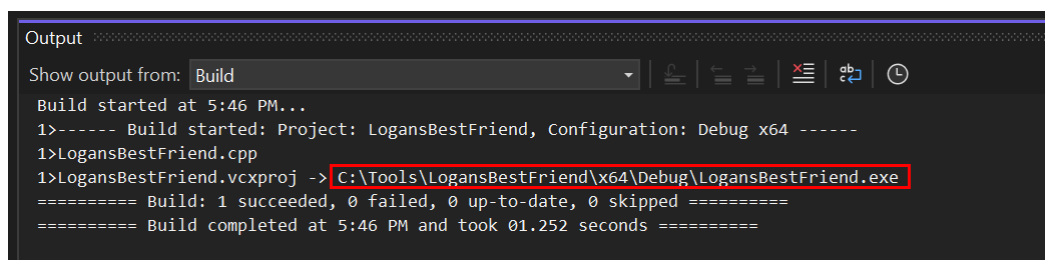
We can generate some stager shellcode through Cobalt strike by going to Payloads -> Stager Payload Generator, selecting C, and saving it to our attacker machine.



We can then edit the file, and copy the shellcode and insert it into LogansBestFriend.

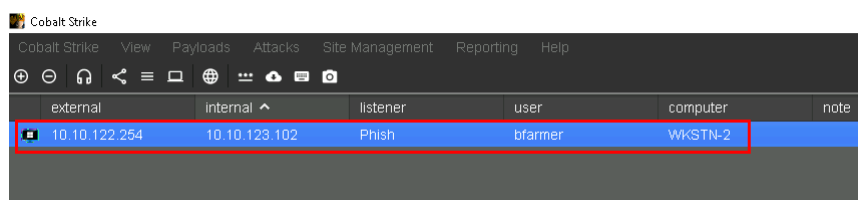


Then, compile the LogansBestFriend payload with our Cobalt strike stager shellcode included



Now once we execute our payload on the target, we should get a callback without alerting Windows Defender.

And boom, looks like we got a callback!:



No popups from Defender, and everything looks good! Navigating to the Windows Security panel, it looks like we've flown directly under defenders radar.

Current threats

No current threats.

Last scan: 1/30/2024 9:22 AM (quick scan)

0 threats found.

Scan lasted 47 seconds

42823 files scanned.

Quick scan

[Scan options](#)

[Allowed threats](#)

[Protection history](#)

“inline-ExecuteAssembly” and the “execute-assembly” dilemma

As I mentioned earlier when referring to the “post-ex” block in the malleable C2 profile, when an operator runs “execute-assembly” it will fork&run the specified process and inject the .NET assembly to avoid dropping potentially signed post exploitation tools to disk. This was very effective until recently, when AV and EDR got really good at detecting anomalous child processes. Seeing a child process spawn, for example: rundll32 with no arguments, and watching it die shortly after is pretty suspicious.

You’ll probably notice that if you try and run an execute-assembly command in our current configuration it will be caught by defender because of that anomalous child process.

So in this “Cat and Mouse Game”, security researchers came up with “inline-ExecuteAssembly”, which is a Beacon Object File (BOF) for Cobalt strike that executes .NET assemblies without the fork&run method.

inline-ExecuteAssembly is publicly available on GitHub, and can be downloaded [here](#)

After downloading and loading the aggressor script into Cobalt strike, you should be able to execute .NET assemblies without the fork&run process, for example:

```
beacon> inlineExecute-Assembly --dotnetassembly /root/Desktop/Seatbelt.exe
```

inlineExecute-Assembly also has a large amount of evasion tools which come bundled, including the “-amsi” flag for AMSI bypasses and the “-etw” for disabling ETW.

Conclusion

Through Cobalt strikes amazing customization, evading modern defenses just requires a local copy of the AV or EDR engine for extensive testing and time. There are 3 main ways in which AV stops malicious applications:

1. Signatures
2. In-memory scanning

3. Behavioral

By ensuring that you have an answer to all 3 defenses, bypassing AV becomes trivial. Cobalt strike's malleable C2 profiles, the artifact kit, custom loaders, and BOF's might be effective right now - but who knows what will be in the future. The field is ever evolving, ever changing, and new mitigations and attack vectors are popping up everywhere. The most important thing to do as a Cybersecurity professional, whether its offense or defense, is to make sure you stay up to date with all the fun, new, and exciting, things in the world of security.